

Memory Leaks in Redux Toolkit

Memory leaks usually happen when:

- Old data stays in Redux store and is never cleared.
- API calls (RTK Query) keep running after a component unmounts.
- Event listeners or subscriptions are not removed.
- Large objects are stored globally in Redux without cleanup.

Common causes

1) Large data stored in Redux and never cleared

Example:

- API returns **1000+ items**.
- You store them in Redux.
- User leaves the page.
- Data is still in Redux.

This unused data stays in memory.

2) RTK Query API calls not handled properly

Memory issues can happen when:

- API keeps refetching
- Cached data is kept for too long
- Component unmounts but data remains

3) Subscriptions and listeners not cleaned

Example:

- WebSocket connection
- Custom listeners
- Timers or intervals

If these are not stopped, memory keeps increasing.

4) Storing wrong things in Redux

Never store:

- DOM elements
- Functions
- Class instances
- Huge objects that are rarely used

Redux is meant only for **serializable state**.

Debug memory leaks in Redux Toolkit

1) Chrome DevTools

Steps:

- Open Chrome DevTools
- Go to **Memory tab**
- Take **Heap Snapshot**
- Navigate pages in your app
- Take another snapshot

- Compare memory usage

If memory keeps increasing -> leak exists

2) Redux DevTools

Check:

- Redux state size
- Same data appearing repeatedly
- Old page data still present

If state grows but never shrinks, that's a problem.

3) Clear Redux state when page unmounts

Very important for:

- Dashboards
- Lists
- Forms
- Reports

Create a `resetState` action and dispatch it when leaving the page.

4) RTK Query best practices

To avoid memory issues:

- Use `skip` when API is not needed
- Reduce `keepUnusedDataFor`
- Don't over-cache data

- Let RTK Query cancel requests automatically on unmount

Author – Swapnil Banta